Resumen del análisis de SonarCloud

Proyecto mateozorzi_practice-sdd · rama main (commit 54b529f) · análisis del 24/10/2026 14:47 (ART) · posterior a los hitos SDD

1. Resumen ejecutivo

El proyecto (≈10,6 k líneas de código C/C++) pasa el Quality Gate (**Passed**) por primera vez. Tiene 23 issues abiertos y 0 security hotspots. La cobertura pasó de 0 % a 90,2 % (94,6 % de líneas y 84,7 % de condiciones) gracias al reporte de gcovr que ahora se envía, y la cobertura del código nuevo es 86,8 % (el gate exige 80 %). La seguridad subió de C a A: los 5 usos de PRNG (S2245) se revisaron, son aleatoriedad del juego y la regla se ignora en la configuración. La mantenibilidad sigue en A (53 min de deuda) y la duplicación bajó a 0,5 %. El punto débil sigue siendo la confiabilidad (rating E): las 4 llamadas a `wait()` sin predicado en `common/queue.h` (Blocker) siguen abiertas y aparecen 10 destructores que Sonar considera que pueden dejar escapar excepciones (S1048). El gate pasa igual porque evalúa solo el código nuevo desde el 19/10, y ahí la confiabilidad es A.

Comparado con el commit pre-SDD (8ae6d5d) analizado con la misma configuración, las 23 issues son exactamente las mismas: el flujo SDD no introdujo ni arregló ninguna. Las mejoras respecto del análisis del 19/10 (seguridad, cobertura, duplicación) vienen de la configuración del análisis y de los tests agregados antes del SDD.

2. Métricas generales

Métrica	Valor	Métrica	Valor
Líneas de código (ncloc)	10.598	Líneas totales	15.883
Archivos	184	Funciones	852
Complejidad ciclomática	1.845	Complejidad cognitiva	1.244
Duplicación	0,5 % (4 bloques)	Comentarios	6,9 %
Cobertura	90,2 %	Deuda técnica	53 min
Cobertura de líneas	94,6 % (328 de 6.069 sin cubrir)	Cobertura de condiciones	84,7 % (753 de 4.915 sin cubrir)
Issues abiertos	23	Security hotspots	0 (100 % revisados)

Lenguajes: C++ 7.246 líneas · C 3.154 (headers .h) · YAML 198.

Alcance del análisis: `sonar.sources` = client, server, common y editor; los tests (`tests/`) se declaran en `sonar.tests` y no suman líneas de código. `installer.sh`, `openspec/` y `.github/` ya no se analizan.

3. Ratings por dimensión

Dimensión	Rating	Issues	Motivo
Seguridad	A	0	Sin vulnerabilidades: S2245 revisado y excluido por configuración.
Confiabilidad	E	14	Al menos un issue de impacto Blocker (S5404 en queue.h).
Mantenibilidad	A	9	Ratio de deuda técnica muy bajo (53 min sobre ~10,6 k LOC).
Revisión de seguridad	A	0 hotspots	No hay hotspots pendientes de revisión.

Quality Gate (Sonar way, código nuevo desde el 19/10/2026): Passed

Condición	Umbral	Valor	Estado
Confiabilidad en código nuevo	A	A	OK
Seguridad en código nuevo	A	A	OK
Mantenibilidad en código nuevo	A	A	OK
Cobertura en código nuevo	≥ 80 %	86,8 %	OK
Duplicación en código nuevo	≤ 3 %	0,0 %	OK
Hotspots revisados	100 %	100 %	OK

4. Issues por regla

Regla	Calidad / severidad	Cant	Archivos	Descripción
cpp:S5404	Confiabilidad / Blocker	4	common/queue.h (L90, 108, 200, 217)	wait() de condition_variable sin predicado.
cpp:S1048	Confiabilidad / High	10	server/: lobby L151, game L74, client_handler L242, game_loop L723, game_monitor L110, receiver L62, sender L63, acceptor L84, server L48; client/receiver.cpp L63	Un destructor puede dejar escapar una excepción.
cpp:S5020	Mantenibilidad / Blocker	3	client/sdl_manager.cpp L33, animations.cpp L93-94	Reemplazar std::rand por <random>.
cpp:S5018	Mantenibilidad / Blocker	4	common/resolver.h L58-59, socket.h L101-102	Move constructor / move assignment sin noexcept.
cpp:S998	Mantenibilidad / Blocker	1	common/thread.h L42	Método llamado main(): se confunde con el entry point global.
cpp:S5502	Mantenibilidad / Blocker	1	client/Lobby/joinlobby.cpp L38	Más de una asignación de recursos en una misma expresión.

Esfuerzo total estimado de remediación: ≈ 7 h 13 min (433 min): 6 h 20 min de confiabilidad y 53 min de mantenibilidad.

5. Comparación con el commit pre-SDD (8ae6d5d)

Para que la comparación sea 1 a 1, se analizó el commit 8ae6d5d (20/10, el último antes de `openspec init`) con el mismo `sonar-project.properties`, la misma base de compilación y la misma cobertura de gcovr. Se publicó en el proyecto aparte mateozorzi_practice-sdd-baseline.

Indicador	Pre-SDD (8ae6d5d)	Post-SDD (54b529f)	Cambio
Quality Gate	No calculado (primer análisis)	Passed	Se habilitó y pasa
Seguridad	A (0 issues)	A (0 issues)	Igual
Confiabilidad	E (14 issues)	E (14 issues)	Igual
Mantenibilidad	A (9 issues, 53 min)	A (9 issues, 53 min)	Igual
Cobertura	89,3 %	90,2 %	+0,9 puntos
Duplicación	0,6 % (4 bloques)	0,5 % (4 bloques)	−0,1 puntos
Líneas de código	9.392	10.598	+1.206
Archivos	171	184	+13
Complejidad ciclomática	1.554	1.845	+291
Complejidad cognitiva	1.033	1.244	+211
Comentarios	6,4 %	6,9 %	+0,5 puntos

Sobre el análisis del 19/10: se hizo con el análisis automático de SonarCloud sobre toda la raíz del repositorio (commit 76847e2), sin base de compilación ni cobertura. Sus diferencias con este informe (seguridad C, 7 issues S7688 de installer.sh, cobertura 0 %, sin S1048 ni S5502) se deben a esa configuración y no al SDD. El análisis de 8ae6d5d con la configuración actual lo confirma: las 10 S1048 y la S5502 ya estaban antes del SDD.

6. Diff de issues a lo largo del flujo SDD

Se cruzaron las issues de los dos análisis por regla, archivo y línea, y los corrimientos de línea se ajustaron con `git diff 8ae6d5d 54b529f`. En el medio hay 48 commits: el pipeline de CI, la inicialización de OpenSpec y los 7 cambios archivados (power-ups Hitos 1–6 y guardado de carreras del editor). Tocan 65 archivos de código (+3.762 / −1.081 líneas).

6.1 Balance

Categoría	Cant	Detalle
Se mantienen	23	Las 23 issues del pre-SDD siguen abiertas, con la misma regla, archivo y severidad.
Arregladas por el SDD	0	Ningún cambio de OpenSpec tenía como objetivo corregir hallazgos de Sonar.
Introducidas por el SDD	0 visibles (1 suprimida)	S2245 en server/power_up.cpp L60, suprimida por configuración (ver abajo).
Total	**Pre: 23 · Post: 23 · Diferencia: 0**	

6.2 Issues que se mantienen y se corrieron de línea

Las 23 issues siguen en el mismo lugar del código. En 4 casos el SDD editó el archivo y solo cambió el número de línea:

Regla	Archivo	Pre → Post	Por qué se corrió
cpp:S1048	server/game_loop.cpp	L678 → L723	Los cambios de power-ups en el game loop agregaron código antes del destructor.
cpp:S1048	server/lobby.cpp	L148 → L151	El constructor recibe la fábrica de generadores de power-ups.
cpp:S1048	server/game.cpp	L73 → L74	El constructor recibe el generador de tipos de power-up.
cpp:S5020	client/sdl_manager.cpp	L32 → L33	Se agregó `#include <iostream>` (verificación del Hito 6).

En estos archivos, y en socket.h y thread.h (donde el SDD solo agregó `override` a los destructores), el flujo editó el mismo archivo que la issue y no la corrigió, porque no estaba en el alcance del cambio.

6.3 Issues introducidas por el SDD

En las issues abiertas no hay ninguna introducida por el SDD, y el Quality Gate lo confirma: confiabilidad, seguridad y mantenibilidad del código nuevo dan A, con 86,8 % de cobertura. El código nuevo de los cambios (power_up.cpp, streets_grid.cpp, la capa de los power-ups, el guardado de carreras del editor) no suma issues.

La única detección nueva es **cpp:S2245 en server/power_up.cpp L60** (`make_random_power_up_type_generator`, Hito 1 de power-ups). Usa `std::mt19937` con `std::random_device` y `uniform_int_distribution`, que es la forma recomendada para S5020, y solo elige el tipo de power-up. Por eso queda suprimida junto con los otros 4 usos. Si la regla se vuelve a activar, aparece como issue del SDD.

6.4 Conclusión para el análisis del flujo

Con la misma configuración, el flujo SDD **no introdujo issues**: agregó 1.206 líneas de código y la cantidad de issues, la deuda técnica y los ratings quedaron iguales. Además subió la cobertura (89,3 % → 90,2 %) y el código nuevo pasa el gate con 86,8 %. Tampoco **arregló ninguna**: las 23 issues previas siguen abiertas, incluso en archivos que el flujo modificó (game_loop.cpp, lobby.cpp, game.cpp, sdl_manager.cpp, socket.h, thread.h). Para que el flujo baje estos números, la remediación tiene que ser un cambio de OpenSpec propio (sección 8), o se puede sumar un control de Sonar en la etapa de verificación de cada cambio para que los archivos tocados salgan sin las issues conocidas.

7. Análisis y recomendaciones

Confiabilidad – S5404 (prioridad alta para subir el rating)

Sigue pendiente desde el análisis anterior. En queue.h el patrón `while (q.size() == max_size) is_not_full.wait(lck);` es funcionalmente correcto, pero Sonar exige la sobrecarga con predicado: `is_not_full.wait(lck, [this]{ return q.size() < max_size || closed; });` (y análogo para `is_not_empty`), sin perder el `throw ClosedQueue()`. Son 4 líneas y quitan el único Blocker de confiabilidad.

Confiabilidad – S1048 en destructores

Los destructores ya envuelven `stop()`/`join()` en `try { ... } catch (const std::exception&)`, pero una excepción que no herede de `std::exception`, o una que lance el propio `std::cerr` dentro del catch, escaparía del destructor y terminaría el programa. Agregar un `catch (...)` final en los 10 destructores cierra las issues; con S5404 y S1048 resueltas, la confiabilidad pasa de E a A.

Mantenibilidad – S5020, S5018, S998 y S5502

Reemplazar `std::rand() %` por `std::uniform_int_distribution` con un `std::mt19937` (como en sounds.cpp y power_up.cpp); agregar `noexcept` a los move de Socket y Resolver; renombrar `Thread::main()` a `thread_main()`, y en joinlobby.cpp L38 crear cada `QTableWidgetItem` en su propia sentencia en lugar de varios `new` dentro de un mismo inicializador de arreglo. En total ≈ 53 min y la deuda técnica queda en 0.

Cobertura

La cobertura llega con `make test` (`gcovr --sonarqube build/coverage.xml`) y excluye las ramas de excepciones que agrega el compilador (`--exclude-throw-branches`, `--exclude-unreachable-branches`). Los archivos con 0 % son puntos de entrada y herramientas: server/main.cpp, client/main.cpp, client/client.cpp y server/tools/streets/ (generador de calles). Conviene excluirlos con `sonar.coverage.exclusions`, ya que no se testean por diseño.

Seguridad – S2245

Se revisaron los 5 usos (tipo de power-up, música de fondo, sala de espera y temblor de cámara): ninguno genera claves, tokens ni valores de seguridad. La regla se ignora en sonar-project.properties con esa justificación, lo que deja la decisión versionada. Si en el futuro se usan números aleatorios con fines de seguridad, hay que volver a activarla.

8. Plan sugerido (orden de impacto)

#	Acción	Issues que cierra	Efecto en ratings	Esfuerzo
1	Predicado en wait() de queue.h	4	Quita el Blocker de confiabilidad	~20 min
2	catch (...) en los 10 destructores	10	Confiabilidad E → A	~30 min
3	<random> en client/, noexcept, renombrar main(), separar los new de joinlobby	9	Mantenibilidad se mantiene A, deuda → 0	~55 min
4	Excluir mains y tools de la cobertura	–	Cobertura más representativa	~5 min

---

Datos obtenidos de la API de SonarCloud (measures, qualitygates, issues, hotspots, project_analyses, project_branches) el 24/10/2026. El análisis corresponde al commit 54b529f de la rama sdd-feature, publicado en SonarCloud como rama main. El análisis pre-SDD de comparación corresponde al commit 8ae6d5d, publicado en el proyecto mateozorzi_practice-sdd-baseline el 25/10/2026 con la misma configuración. La sección 6 cruza las issues de ambos con `git diff 8ae6d5d 54b529f`.